

 Object-Oriented Programming Project

Pharmacy Management System

An OOP-driven desktop and console architecture for automated inventory control, supplier tracking, and secure point-of-sale transactions.

UNIVERSITY

Cairo University

FACULTY

Faculty of Regional & Urban Planning

ACADEMIC YEAR

2025 / 2026

SYSTEM FOUNDATION

Project Overview & Purpose



Core Application

Dual desktop and console architecture engineered strictly with Object-Oriented Programming in Python.



Problem Solver

Eliminates manual inventory tracking errors, stockouts, and accidental sales of expired medications.



Target Operators

Tailored for pharmacy managers, store administrators, and sales staff managing customer checkouts.

OPERATIONAL CHALLENGES

Problem Statement & Solution

Traditional Bottlenecks

- ⚠ Human calculation errors in manual receipt logs.
- ⚠ Undetected expired products reaching customers.
- ⚠ Unnoticed stockouts leading to lost revenue opportunities.

Automated Software Solution

- ✓ Encapsulated business logic enforcing safety validations.
- ✓ Automated stock deduction and real-time expiration checks.
- ✓ Instant reporting on revenue and inventory thresholds.

PROJECT GOALS

Key System Objectives

01

Automation

Streamline inventory, customer records, and checkout calculations.

02

Data Integrity

Organize records securely using dictionaries and collection models.

03

Dual Interface

Offer both an interactive console menu and a Tkinter desktop GUI.

04

OOP Practice

Demonstrate inheritance, polymorphism, and exception handling.

CAPABILITIES

Comprehensive System Features

CRUD Operations

Full management for medicines, categories, suppliers, and customers.

Dynamic stock price and active status modifications.

Smart Search & Filter

Rapid lookups by ID, partial name, or category.

Instant filtering for low-stock and expired products.

Sales & Safety

Automatic stock deduction upon checkout.

Blocks sales of expired or out-of-stock items.

TECH STACK

Technologies & Architecture

Python 3.x

Core programming language driving object models and collections.

Tkinter & TTK

Graphical user interface framework for desktop user interactions.

Datetime & Collections

Standard libraries handling expiration dates and analytics.

CORE OOP PILLARS

Object-Oriented Programming Principles

Encapsulation

Private attributes protected via property getters and validators.

Inheritance

`Customer` and `Supplier` inherit shared traits from `Person`.

Polymorphism

Overridden `display\_info()` methods across person types.

Composition

`Sale` objects contain structured collections of `SaleItem`.

CLASS DESIGN

System Class Breakdown

Person & Subclasses

Base `Person` class extended by `Customer` and `Supplier` for profile management.

Medicine & Category

Manages stock quantities, pricing, expiration dates, and category links.

Pharmacy Manager

Central controller governing all entity dictionaries, CRUD, and reporting.

ARCHITECTURE

Object Relationships

Inheritance (Is-A)

Customer and Supplier inherit shared attributes and validation from Person.

Aggregation (Has-A)

Pharmacy manager aggregates collections of medicines, categories, and sales.

Composition

A Sale transaction is composed of one or more SaleItem line entries.

CODE IMPLEMENTATION

Encapsulation & Property Validation

```
@property
def price(self):
    return self.__price

@price.setter
def price(self, value):
    if float(value) <= 0:
        raise ValueError("Error: Medicine price must be positive.")
    self.__price = float(value)
```

Private attributes prevent invalid direct modification, ensuring financial and inventory data integrity.

CODE IMPLEMENTATION

Inheritance & Method Overriding

```
class Customer(Person):  
    def __init__(self, customer_id, name, phone):  
        super().__init__(customer_id, name, phone)  
  
    def display_info(self):  
        return f"Customer [{self.customer_id}] {self.name} | Phone: {self.phone}"
```

Eliminates code duplication while allowing specialized string formatting for customers and suppliers.

BUSINESS LOGIC

Two-Pass Sales Validation

```
def create_sale(self, sale_id, customer_id, item_requests):
    # Pass 1: Validate expiration, active status, and stock
    for med_id, qty in item_requests:
        med = self._get_or_error(self.medicines, med_id, "Medicine")
        if med.is_expired() or not med.active or qty > med.quantity:
            raise ValidationError("Sale validation failed.")
    # Pass 2: Deduct stock and commit transaction
    for med, qty in checked_items:
        med.reduce_stock(qty)
```

EXECUTION FLOW

End-to-End System Workflow



1. User Interaction

Console menu or Tkinter GUI input.

2. Validation

Checking IDs, pricing, and expiration.



3. Business Logic

Pharmacy manager rule verification.



4. State Update

Stock reduction & report generation.

DESKTOP GUI

Graphical User Interface Design

Inventory & Search Tab

Real-time inventory table view with filters for low-stock and expired items.

Management Forms

Intuitive input forms for adding medicines, categories, suppliers, and customers.

Sales & Reports Tab

Cart checkout interface and dynamic sales revenue summaries.

EVALUATION

Advantages & Development Challenges

OOP Advantages

- ✓ High reusability through inheritance.
- ✓ Strict data integrity via encapsulation.
- ✓ Clean modular architecture.

Key Challenges

- Structuring complex class associations.
- Implementing safe multi-pass sales validation.
- Synchronizing GUI treeviews with dictionaries.

LOOKING AHEAD

Future Enhancements & Conclusion

Database Integration

Replace in-memory dictionaries with SQLite and SQLAlchemy ORM.

Hardware Support

Integrate barcode scanners and automated supplier email alerts.

Conclusion

Demonstrates robust application of OOP principles to solve real-world problems.